

Introduction to IA-32 Instruction Encoding

1. Introduction to IA-32 Instruction Encoding

IA-32 (x86-32) machine instructions are variable-length, built from an ordered sequence of optional and mandatory byte fields. Getting comfortable with how each field is encoded is really the foundation for anything lower-level — writing an assembler, a disassembler, or even just a debugger that needs to make sense of raw instruction bytes.

An instruction is put together from up to five pieces, always in the same order: the opcode, the ModR/M byte, the SIB byte, a displacement, and an immediate value. Not every instruction uses every field — the opcode itself decides which of the remaining fields actually show up.

2. Instruction Byte Diagram

OPCODE | MOD R/M | SIB | DISPLACEMENT | IMMEDIATE

- OPCODE — the primary opcode, 1 to 3 bytes long.
- MOD R/M — 1 byte, present when the instruction touches a register or memory operand.
- SIB (Scale-Index-Base) — 1 byte, only shows up for certain memory addressing forms.
- Displacement — 1, 2, or 4 bytes, present when the addressing form needs a constant offset.
- Immediate — 1, 2, or 4 bytes, present when the instruction carries a literal operand value.

3. Opcode Structure

The ModR/M byte breaks down into three bit fields that together pick the addressing mode and register:

MOD	REG/OPCODE	R/M
2 bits	3 bits	3 bits

The SIB byte follows the same pattern, used for scaled-index memory addressing:

Scale	Index	Base
2 bits	3 bits	3 bits

The Scale field just picks a multiplier for the index register:

Scale	Multiplier
00	x1
01	x2

10	x4
11	x8

4. Opcode Table

These are the instruction forms and opcode encodings this project relies on:

Instruction	Operand Form	Opcode
MOV	r/m32, imm32	C7 /0 id
MOV	r/m32, r32	89 /r
ADD	r/m32, imm8	83 /0 ib
ADD	r/m32, imm32	81 /0 id
ADD	r/m32, r32	01 /r
SUB	r/m32, imm8	83 /5 ib
SUB	r/m32, imm32	81 /5 id
SUB	r/m32, r32	29 /r
DIV	r/m32	F7 /6
INC	r/m32	FF /0
DEC	r/m32	FF /1
PUSH	r32	50+rd
PUSH	r/m32	FF /6
POP	r32	58+rd
POP	r/m32	8F /0
RET	-	C3

5. 32-bit Register Encoding Table

How the REG and R/M bit patterns map onto the eight 32-bit general-purpose registers:

REG	Register
000	EAX
001	ECX
010	EDX
011	EBX
100	ESP
101	EBP
110	ESI
111	EDI

32-bit addressing forms with the ModR/M byte

Mod = 00 — memory, no displacement:

Effective Address\ R/M	000	001	010	011	100	101	110	111	
[EAX]	000	00	08	10	18	20	28	30	38
[ECX]	001	01	09	11	19	21	29	31	39
[EDX]	010	02	0A	12	1A	22	2A	32	3A
[EBX]	011	03	0B	13	1B	23	2B	33	3B
[--] [--]	100	04	0C	14	1C	24	2C	34	3C
disp32	101	05	0D	15	1D	25	2D	35	3D
[ESI]	110	06	0E	16	1E	26	2E	36	3E
[EDI]	111	07	0F	17	1F	27	2F	37	3F

Mod = 01 — memory + 8-bit displacement:

Effective Address\ R/M	000	001	010	011	100	101	110	111	
[EAX] +disp8	000	40	48	50	58	60	68	70	78
[ECX] +disp8	001	41	49	51	59	61	69	71	79
[EDX] +disp8	010	42	4A	52	5A	62	6A	72	7A
[EBX] +disp8	011	43	4B	53	5B	63	6B	73	7B
[--] [--] +disp8	100	44	4C	54	5C	64	6C	74	7C
[EBP] +disp8	101	45	4D	55	5D	65	6D	75	7D
[ESI] +disp8	110	46	4E	56	5E	66	6E	76	7E
[EDI] +disp8	111	47	4F	57	5F	67	6F	77	7F

Mod = 10 — memory + 32-bit displacement:

Effective Address\ R/M	000	001	010	011	100	101	110	111	
[EAX] +disp32	000	80	88	90	98	A0	A8	B0	B8
[ECX]	001	81	89	91	99	A1	A9	B1	B9

+disp3 2									
[EDX] +disp3 2	010	82	8A	92	9A	A2	AA	B2	BA
[EBX] +disp3 2	011	83	8B	93	9B	A3	AB	B3	BB
[--] [--] +disp3 2	100	84	8C	94	9C	A4	AC	B4	BC
[EBP] +disp3 2	101	85	8D	95	9D	A5	AD	B5	BD
[ESI] +disp3 2	110	86	8E	96	9E	A6	AE	B6	BE
[EDI] +disp3 2	111	87	8F	97	9F	A7	AF	B7	BF

Mod = 11 — register-direct:

Effective Address\ R/M	000	001	010	011	100	101	110	111	
EAX	000	C0	C8	D0	D8	E0	E8	F0	F8
ECX	001	C1	C9	D1	D9	E1	E9	F1	F9
EDX	010	C2	CA	D2	DA	E2	EA	F2	FA
EBX	011	C3	CB	D3	DB	E3	EB	F3	FB
ESP	100	C4	CC	D4	DC	E4	EC	F4	FC
EBP	101	C5	CD	D5	DD	E5	ED	F5	FD
ESI	110	C6	CE	D6	DE	E6	EE	F6	FE
EDI	111	C7	CF	D7	DF	E7	EF	F7	FF

32-bit addressing forms with the SIB byte

SIB Index table:

Bits	Register
000	EAX
001	ECX
010	EDX
011	EBX

100	No index
101	EBP
110	ESI
111	EDI

SIB Base table:

Bits	Register
000	EAX
001	ECX
010	EDX
011	EBX
100	ESP
101	*EBP
110	ESI
111	EDI

6. Opcode Format Notation (/r, /0, /5, +rd)

- /r — the ModR/M byte's REG field holds an actual register operand (source or destination), covering register-to-register and register-to-memory forms.
- /digit (e.g. /0, /5, /6) — here REG isn't a register at all, it's a fixed 3-bit opcode extension. The digit listed is the exact bit pattern REG must carry for that instruction — it's how one opcode byte (like 81 or FF) can stand in for several different instructions, distinguished only by those REG bits.
- +rd — the register is folded directly into the low 3 bits of the opcode byte itself, skipping ModR/M entirely. Add the register's 3-bit code (0–7) to the base opcode value to get the final byte. Used by the short forms PUSH r32 (50+rd) and POP r32 (58+rd).
- ib / id — a suffix flagging that an immediate follows: ib means a 1-byte immediate, id means a 4-byte (doubleword) immediate.

7. Verify / Add Opcodes Required by the Final Code Snippet

This section is left open until the final code snippet is available — at that point it's worth going back through the opcode table in Section 4 and checking it against what the snippet actually emits:

- Confirm the operand form (register / memory / immediate) matches a row already in Section 4.
- If something's missing, add it, using the SDM as the source of truth for the opcode.
- Recompute the ModR/M (and SIB, where relevant) byte values for each operand combination in use, and cross-check them against the addressing-form tables in Section 5.

Once that snippet is available, this section can be filled in with the concrete opcode list it needs.